

Formation Complète .NET

Les Bases : C# et SQL

Socle Théorique et Fondations du Langage

Objectifs du Module

- Comprendre l'architecture de la plateforme .NET et le fonctionnement du runtime (CLR).
- Installer et configurer un environnement de développement professionnel (Visual Studio / VS Code).
- Maîtriser la syntaxe fondamentale du langage C# et ses structures de contrôle.
- Manipuler avec aisance les collections et structures de données (tableaux, listes, dictionnaires).
- Appréhender les bases de la Programmation Orientée Objet (classes, interfaces, héritage).
- Concevoir et modéliser des bases de données relationnelles avec SQL Server.
- Acquérir une autonomie complète avant d'aborder les projets complexes d'ingénierie logicielle.

Support Pédagogique

Édition Académique & Technique — Version .NET 8+

Kevin Özkaraca

Ebéniste spécialisé : C# - SQL - ASP .NET Core - Angular - WPF - MAUI

1^{er} août 2026

1 Environnement et Premier Programme

1.1 Installation de l'Environnement de Développement

Pour développer efficacement en C#, le choix et la configuration de l'Environnement de Développement Intégré (IDE) sont cruciaux. Deux options principales s'offrent au développeur :

1.1.1 Visual Studio (Environnement Complet)

Visual Studio est l'IDE de référence pour le développement .NET sous Windows.

1. Télécharger la version **Visual Studio Community** (gratuite pour l'apprentissage et l'usage individuel).
2. Lors de l'installation via le *Visual Studio Installer*, cocher impérativement la charge de travail (*workload*) :

Développement .NET Desktop (comprend le SDK .NET, les outils de débogage et C#).

1.1.2 Visual Studio Code (Éditeur Léger & Multiplateforme)

VS Code est un éditeur de code extensible, idéal pour les environnements Linux, macOS ou Windows légers.

1. Télécharger et installer le **SDK .NET 8+** depuis le site officiel Microsoft.
2. Installer **Visual Studio Code**.
3. Ouvrir l'onglet Extensions () et installer la suite d'extensions officielles :
 - **C# Dev Kit** (fournit IntelliSense, la gestion de solutions et le débogage avancé).
 - **C#** (support du langage via OmniSharp/Roslyn).

1.2 Qu'est-ce que C#, .NET, CLI et CLR ?

Pour programmer en C# avec rigueur, il est essentiel d'assimiler l'architecture de la plateforme.

1.2.1 Vue d'Ensemble de l'Écosystème

Tableau : Composants clés de l'écosystème Microsoft .NET

Terme	Nature	Description
C#	Langage	Langage de programmation moderne, orienté objet, à typage fort.
.NET	Plateforme	Écosystème réunissant le runtime, les bibliothèques standard et les outils.
Roslyn	Compilateur	Compilateur officiel traduisant le C# en langage intermédiaire (CIL).
CLI	Norme	<i>Common Language Infrastructure</i> — Norme (ECMA-335) régissant .NET.
CLR	Runtime	<i>Common Language Runtime</i> — Moteur d'exécution virtuel exécutant le code.

1.2.2 Définitions des Concepts Clés

Afin de bien appréhender l'écosystème .NET, il est crucial de comprendre la terminologie employée :

- **Langage typé vs non typé (Typage fort)** : Un **langage typé** (comme C# ou Java) exige que chaque donnée en mémoire possède une nature stricte (entier, texte, booléen) définie à l'avance. À l'inverse, un langage à **typage dynamique** (souvent appelé vulgairement "non typé", comme JavaScript ou Python) permet de changer la nature d'une variable à la volée. C# est un langage **fortement typé** : le compilateur vérifie la cohérence des opérations et interdit les actions illogiques (ex : diviser un texte par un entier) avant même l'exécution, garantissant ainsi une grande fiabilité et robustesse des applications.
- **Runtime (Moteur d'exécution)** : Un *runtime* est un environnement logiciel qui s'exécute en arrière-plan et héberge votre programme. Sans le runtime (le CLR pour .NET), votre code compilé serait incompréhensible pour le système d'exploitation. Le runtime est responsable de la traduction finale des instructions pour le processeur physique, de l'allocation mémoire et de la sécurité logicielle.
- **Bibliothèque (Library)** : Une bibliothèque est une collection de code pré-écrit, optimisé et testé, prêt à être intégré dans vos propres projets pour vous éviter de "réinventer la roue". La plateforme .NET inclut la *Base Class Library* (BCL), offrant des milliers de fonctionnalités prêtes à l'emploi (manipulation de fichiers, connexions réseau, algorithmes cryptographiques).
- **Outils (Tools)** : Ce sont les utilitaires qui accompagnent le développeur tout au long du cycle de production logicielle. Quelques exemples incontournables dans .NET :
 - **NuGet** : Le gestionnaire de paquets officiel, permettant de rechercher, télécharger et intégrer des bibliothèques externes tierces.
 - **CLI dotnet (Command Line Interface)** : L'outil permettant de créer, compiler, tester et déployer des applications .NET depuis un terminal.
 - **MSBuild** : Le moteur de construction sous-jacent qui orchestre la compilation des solutions complexes.
- **Norme (Standard / Spécification)** : Une norme est un document théorique et officiel (tel un cahier des charges). Elle ne contient aucun code exécutable, mais dicte les règles absolues de fonctionnement d'un système. Par exemple, la **CLI** est la norme (le plan de construction architectural), tandis que le **CLR** est l'implémentation de cette norme (le moteur réel construit en respectant rigoureusement ce plan).

1.2.3 La CLI (Common Language Infrastructure) : La Norme Théorique

La **CLI** (*Common Language Infrastructure*) est une spécification internationale qui décrit l'architecture nécessaire pour permettre à plusieurs langages haute performance (C#, F#, VB.NET) de s'exécuter sur la même plateforme sans recompilation spécifique à la machine hôte.

La CLI définit deux piliers fondamentaux :

1. **CIL / IL (Common Intermediate Language)** : Le jeu d'instructions universel auquel tous les langages .NET se réduisent après la première phase de compilation.
2. **CTS (Common Type System)** : La spécification unifiée des types de données (garantissant qu'un entier possède la même représentation mémoire en C# et en F#).

1.2.4 Le CLR (Common Language Runtime) : Le Moteur d'Exécution

Le **CLR** est l'implémentation concrète de la spécification CLI par Microsoft. Il agit comme une machine virtuelle hautement optimisée qui prend en charge :

- **La Compilation JIT (Just-In-Time)** : Traduction à la volée du code IL en instructions machines natives (0 et 1) adaptées au processeur cible (x64, ARM64).
- **Le Garbage Collector (GC)** : Gestion automatique de la mémoire et libération des objets inutilisés.
- **La Sécurité de Typage** : Vérification que le code n'accède pas à des zones mémoire non autorisées.

1.2.5 Les 4 Piliers de l'Architecture .NET

Voici le rôle exact de chaque composant dans le cycle de vie de votre code :

- **La CLI (L'Infrastructure)** : C'est le cahier des charges. Elle dicte les règles que tout langage .NET doit respecter (les types de données, la structure du code). Elle garantit que le code C# pourra cohabiter avec du code F#.
- **Roslyn (Le Compilateur)** : C'est l'outil qui lit votre code source C# (). Il vérifie qu'il n'y a pas d'erreurs de syntaxe, puis le traduit en **Code IL** (un code intermédiaire compréhensible par la machine virtuelle).
- **Le CLR (Le Moteur d'Exécution)** : C'est le cœur de .NET. Il prend le Code IL généré par Roslyn et le fait tourner. Il s'occupe de gérer la mémoire (Garbage Collector) et d'assurer la sécurité du programme pendant qu'il tourne.
- **Le JIT (Le Traducteur Final)** : C'est un composant interne au CLR. Juste avant l'exécution, il traduit le Code IL en instructions binaires ultra-rapides, spécifiquement optimisées pour le processeur de la machine sur laquelle le programme tourne (Intel, AMD, ARM).

1.3 Structure d'un Programme C#

Tout programme C# s'articule autour de trois éléments fondamentaux : l'espace de noms (), la classe (), et le point d'entrée ().

1.3.1 Structure Classique (Explicite)

Voici le squelette standard d'une application C# console :

```

1
2
3
4
5
6
7
8
9
10
11
12
13

```

Listing 1 – Structure classique d'un fichier C# (Program.cs)

Explication détaillée du code :

- : En C#, le code est organisé en bibliothèques (espaces de noms). Cette ligne dit au compilateur : *"Je veux utiliser les outils fournis par Microsoft, notamment ceux pour écrire dans la console"*. Sans cette ligne, il faudrait écrire .

- : Un (espace de noms) est un dossier virtuel. Il permet de ranger vos classes pour que votre code ne se mélange pas avec celui d'autres développeurs. Si deux classes s'appellent , le namespace permet de les différencier.
- : En C# classique, tout doit obligatoirement être contenu dans une "classe". La classe est traditionnellement le point de départ. Le mot-clé signifie que cette classe n'est visible que dans ce projet.
- : C'est le point d'entrée critique de l'application. Quand vous lancez le programme, le moteur de .NET cherche très exactement cette méthode et exécute le code qui se trouve à l'intérieur de ses accolades.

1.3.2 Structure Moderne : Top-Level Statements (C# 9+ / .NET 8)

Dans les versions modernes de C#, le boilerplate (code répétitif) peut être omis pour les scripts simples et les points d'entrée :

1
2

Listing 2 – Structure moderne avec Top-Level Statements

Quelle structure choisir et comment l'afficher ?

Aujourd'hui, **Microsoft recommande d'utiliser les Top-Level Statements** (la structure moderne) pour les nouveaux projets. C'est plus clair et cela enlève le code inutile pour démarrer. C'est le comportement par défaut quand on crée un nouveau projet .NET. Cependant, Roslyn cache simplement l'ancienne structure : il génère toujours une classe et une méthode en arrière-plan pendant la compilation.

Comment revenir à la structure classique ? Si votre projet nécessite une configuration complexe au démarrage ou si vous préférez l'ancienne méthode, vous pouvez forcer la structure classique :

- Dans Visual Studio, lors de la création du projet, cochez : *"Do not use top-level statements"*.
- En ligne de commande (CLI), tapez : .

1.4 Le Cycle de Compilation et d'Exécution

Le processus complet de transformation du code source C# en exécution système suit un pipeline strict en deux phases :

1. Phase 1 : Compilation Statique (Roslyn)

- Analyse syntaxique et sémantique du fichier .
- Génération de l'assemblage (fichier ou).
- L'assemblage contient le **Code IL** et les **Métadonnées**.

2. Phase 2 : Exécution Dynamique (CLR / JIT)

- Chargement de l'assemblage par le CLR.
- Le compilateur ****JIT**** traduit les blocs IL nécessaires en instructions machines natives.
- Le processeur exécute le code natif.

1.5 Synthèse du Chapitre

Tableau : Récapitulatif des notions clés du Chapitre 1

Concept	A retenir
Roslyn	Compilateur C# qui transforme le code en IL .
Code IL	Langage intermédiaire universel de la plateforme .NET.
CLR	Moteur d'exécution qui gère la mémoire (GC) et compile le code IL via le JIT .
Main()	Point d'entrée obligatoire de toute application console C#.

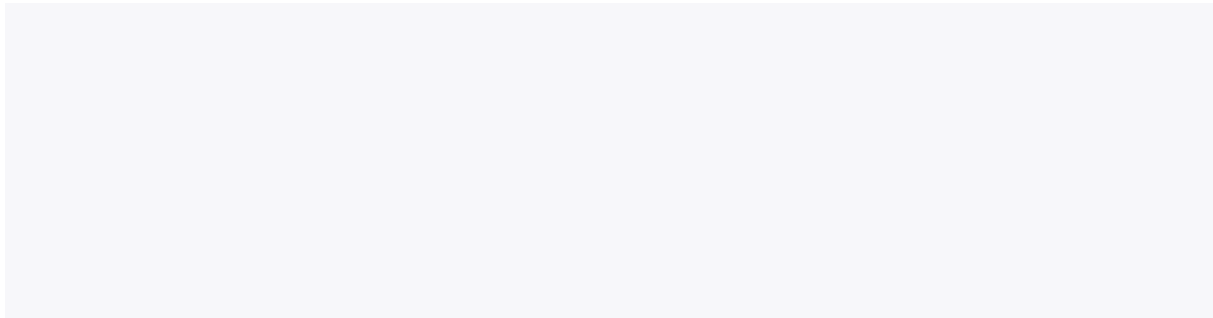
2 Variables et Types de Données

Les variables constituent le fondement de la programmation impérative. Elles permettent d'allouer de l'espace mémoire pour stocker, lire et manipuler des données tout au long de l'exécution d'un programme. En C#, langage à typage statique fort, chaque variable doit être déclarée avec un type précis, garantissant la sûreté du code dès la compilation.

2.1 Déclaration et Initialisation

La déclaration d'une variable exige de spécifier son **type** suivi de son **identifiant** (nom). L'initialisation consiste à lui attribuer une valeur en mémoire via l'opérateur d'affectation .

1
2
3
4
5
6
7
8
9
10



Listing 3 – Déclaration et initialisation standard

2.2 Les Types de Données Primitifs

Le système unifié des types (CTS - Common Type System) de .NET propose de multiples types adaptés aux différents besoins de précision, d'espace mémoire et de logique métier.

2.2.1 Types Fondamentaux

Tableau : Les types de données les plus courants

Type	Nature	Exemple d'utilisation
	Entier signé 32-bit	
	Chaîne de caractères (UTF-16)	
	Booléen (logique)	
	Décimal (64-bit IEEE 754)	

2.2.2 Types Spécifiques et Précision

Pour des besoins d'ingénierie particuliers, C# propose des types avec des capacités mémoire étendues ou des gestions d'arrondis spécifiques. Le suffixe de type (,) est obligatoire lors de l'affectation littérale.

Tableau : Types avancés et suffixes

Type	Spécificité	Exemple
	Haute précision	
	Simple précision (32-bit)	
	Grand entier (64-bit)	
	Caractère unique	

Critères de choix : float, double et decimal

- : Utilisé principalement pour la 3D, le rendu graphique ou le traitement du signal (la vitesse de calcul CPU/GPU prime sur l'exactitude extrême).
- : Type décimal par défaut en C#, idéal pour les calculs mathématiques et la trigonométrie.
- : Implémente une arithmétique en base 10. Strictement indispensable pour les calculs monétaires et financiers afin d'éviter les pertes de précision liées au format flottant binaire.

2.3 Inférence de Type avec

L'instruction `var` demande au compilateur de déduire automatiquement le type de la variable en analysant l'expression à droite de l'opérateur d'affectation. Cela allège l'écriture sans sacrifier la sécurité.

1
2
3
4
5

Listing 4 – Typage statique implicite avec var

Avertissement Architectural : `string` n'est pas dynamique

Le mot-clé `string` maintient un typage **statique fort**. Lors de la compilation, l'instruction `string s = "Hello World";` est transformée en `string s = "Hello World";`. Il sera donc impossible d'affecter une chaîne de caractères à `string` ultérieurement.

2.4 Les Constantes (`const`)

Une constante alloue un espace mémoire immuable. Sa valeur est résolue et scellée lors de la compilation. Elle ne pourra jamais être modifiée durant l'exécution, garantissant ainsi l'intégrité de la donnée.

1
2

Listing 5 – Déclaration de constantes

2.5 Manipulation des Chaînes de Caractères

En C#, les chaînes (`string`) peuvent être combinées (concaténation) à l'aide de l'opérateur `+`. Notez que l'API de base permet également de lire les entrées de l'utilisateur.

1
2
3
4
5
6
7
8
9

Listing 6 – Concaténation et récupération d’entrées

2.6 La Portée des Variables (Scope)

La portée d’une variable définit son accessibilité et sa durée de vie en mémoire. C# définit des périmètres d’accès très stricts, indispensables à l’encapsulation.

Tableau : Les niveaux de portée en C#

Type de portée	Zone de déclaration	Durée de vie
Locale	Dans une méthode ou un bloc	Détruite à la fin du bloc d’exécution.
Champ d’instance	Dans la classe (hors méthode)	Détruite avec l’instance de l’objet (via le GC).
Statique	Dans la classe (avec)	Réside en mémoire jusqu’à la fermeture du processus.

2.6.1 Démonstration des Portées et du Masquage (Shadowing)

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29

Listing 7 – Gestion de la portée, variables statiques et masquage

2.7 Le Type

Contrairement à `unsafe`, le mot-clé `dynamic` désactive intégralement la vérification statique du compilateur. Le type n'est résolu qu'au moment de l'exécution (Runtime).

1
2

Listing 8 – Désactivation du typage avec `dynamic`

L'usage de `dynamic` doit être rigoureusement limité à des cas d'ingénierie spécifiques (interopérabilité COM, réflexion, traitement de structures JSON non typées) car il annule toute la sécurité offerte par le langage.

2.8 Conversion de Types (Casting)

La conversion de types est une opération courante, notamment lors de l'intégration de flux de données externes.

1
2
3
4
5
6
7
8
9
10
11

Listing 9 – Mécanismes de conversion en C#

2.9 Bonnes Pratiques de Codage

Règles strictes de développement C#

- **Variables non initialisées** : Le compilateur C# refuse catégoriquement l'utilisation d'une variable locale non assignée (`?`). Les champs de classe, eux, prennent par défaut la valeur `0` ou `default`.
- **Confusion Affectation / Comparaison** : Ne jamais confondre l'opérateur d'affectation `=` avec l'opérateur d'égalité logique `==`.
- **Déclarations en ligne** : L'écriture `var` est permise par le compilateur mais prohibée par les conventions de Clean Code. L'architecture logicielle exige une déclaration par ligne pour préserver la lisibilité de l'historique de versionnage (Git).

3 Conventions de Nommage, Formatage et Clean Code

Écrire du code informatique ne se limite pas à produire une séquence d'instructions exécutables par le compilateur. Dans l'ingénierie logicielle professionnelle, le code C# doit être **lisible**, **maintenable** et **homogène**. Les conventions de style constituent le langage commun facilitant la collaboration au sein des équipes de développement.

P1_02_b.tex

3.1 Conventions de Nommage Officielles

Le nommage d'un élément logiciel doit rendre le code auto-documenté. Microsoft définit des règles précises de casse selon la nature et la portée de chaque symbole.

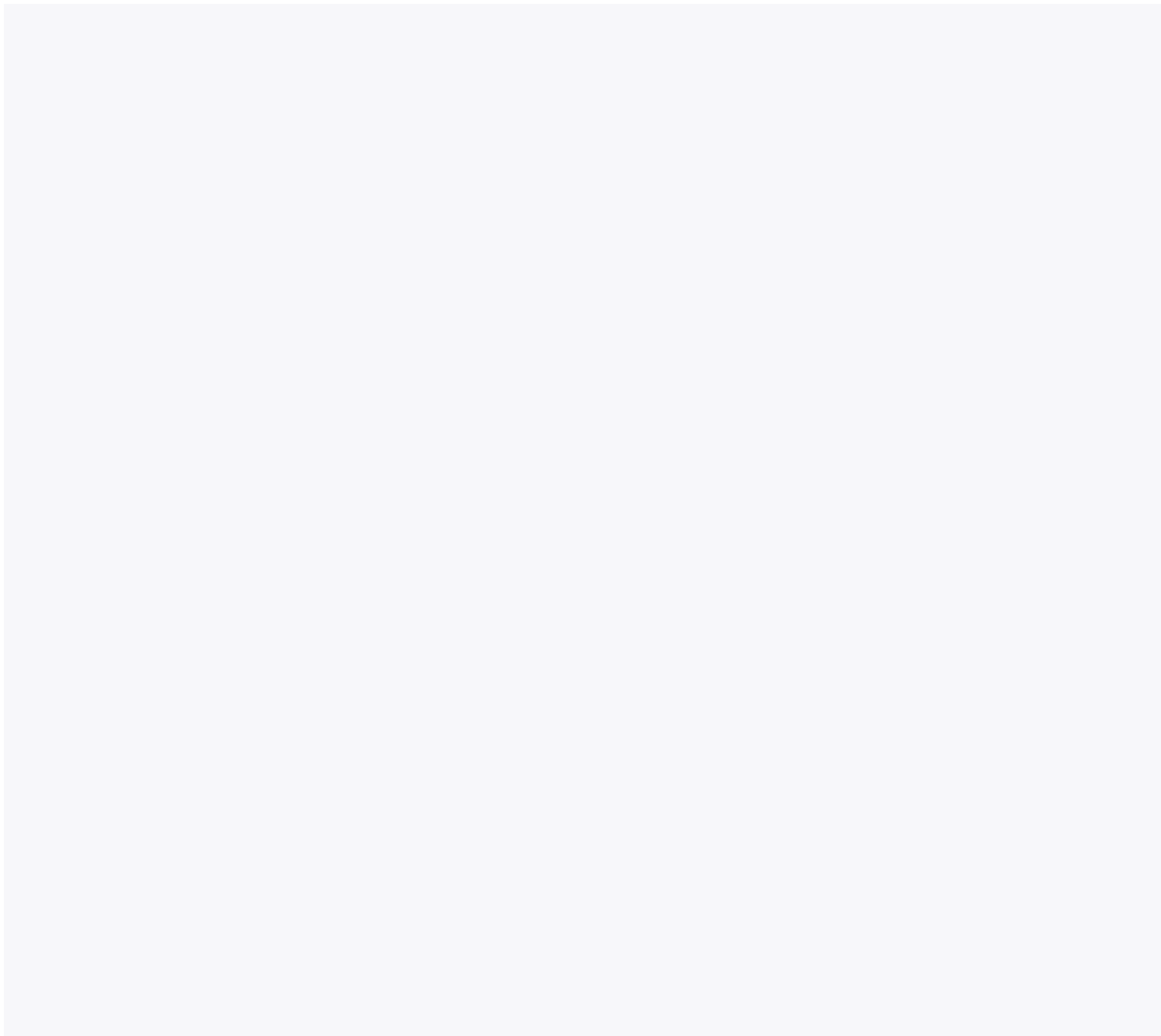
3.1.1 Les Styles de Casse en C#

Tableau : Règles de casse officielles C# (Microsoft Design Guidelines)

Style	Format	Exemple	Cas d'utilisation
PascalCase	Majuscule à chaque mot		Classes, structures, enums, méthodes, propriétés, événements.
camelCase	1 ^{er} mot en minuscule		Variables locales, paramètres de méthodes.
_camelCase	camelCase précédé de		Champs privés d'une classe (<i>private fields</i>).
IPascalCase	PascalCase avec préfixe		Interfaces.
PascalCase	Majuscule à chaque mot		Constantes (et).

3.1.2 Exemples de Conformité Syntaxique

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34



Listing 10 – Conventions de nommage recommandées en C#

Pièges de nommage à éviter

- **Ne pas utiliser le `()`** : réservé à d'autres langages (Python, Rust).
- **Éviter la notation hongroise `(, )`** : le typage C# est déjà statique et explicite.
- **Omettre le préfixe `sur les interfaces`** : rend la distinction impossible avec les classes abstraites.

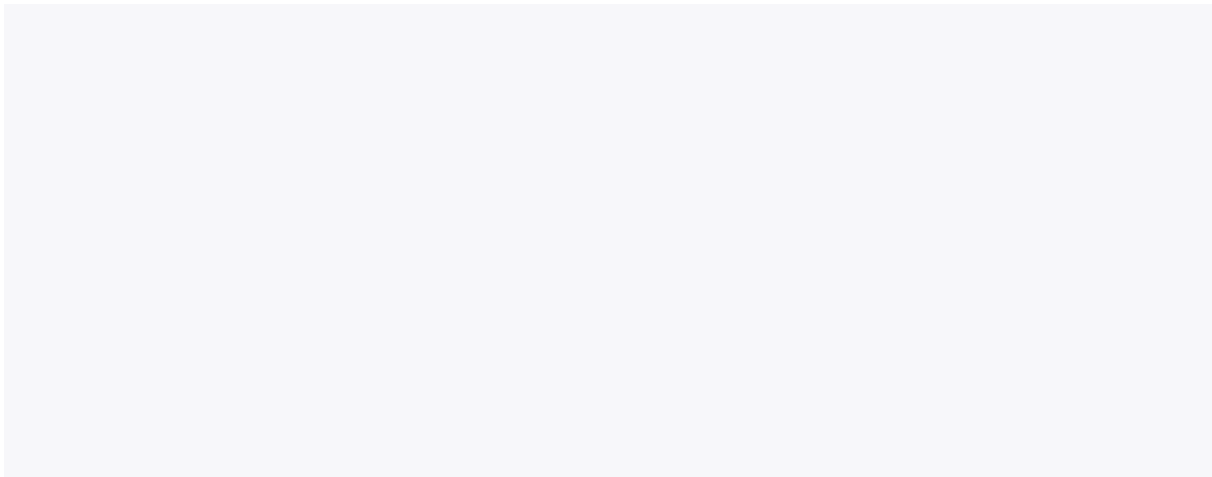
3.2 Conventions de Formatage (Style Allman)

Le formatage régit l'organisation visuelle du code source. C# adopte historiquement le style d'accolades **Allman**.

3.2.1 Règles Fondamentales d'Organigramme Visuel

1. **Accolades Allman** : L'accolade ouvrante se place **toujours sur sa propre ligne**, alignée verticalement avec la structure parente.
2. **Indentation** : Strictement 4 espaces par niveau de bloc (ne pas mélanger tabulations et espaces).
3. **Espacement des opérateurs** : Un espace unique de part et d'autre des opérateurs binaires `(, , , )`.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15



Listing 11 – Comparaison de formatage : Allman (C#) vs K&R (Java/JS)

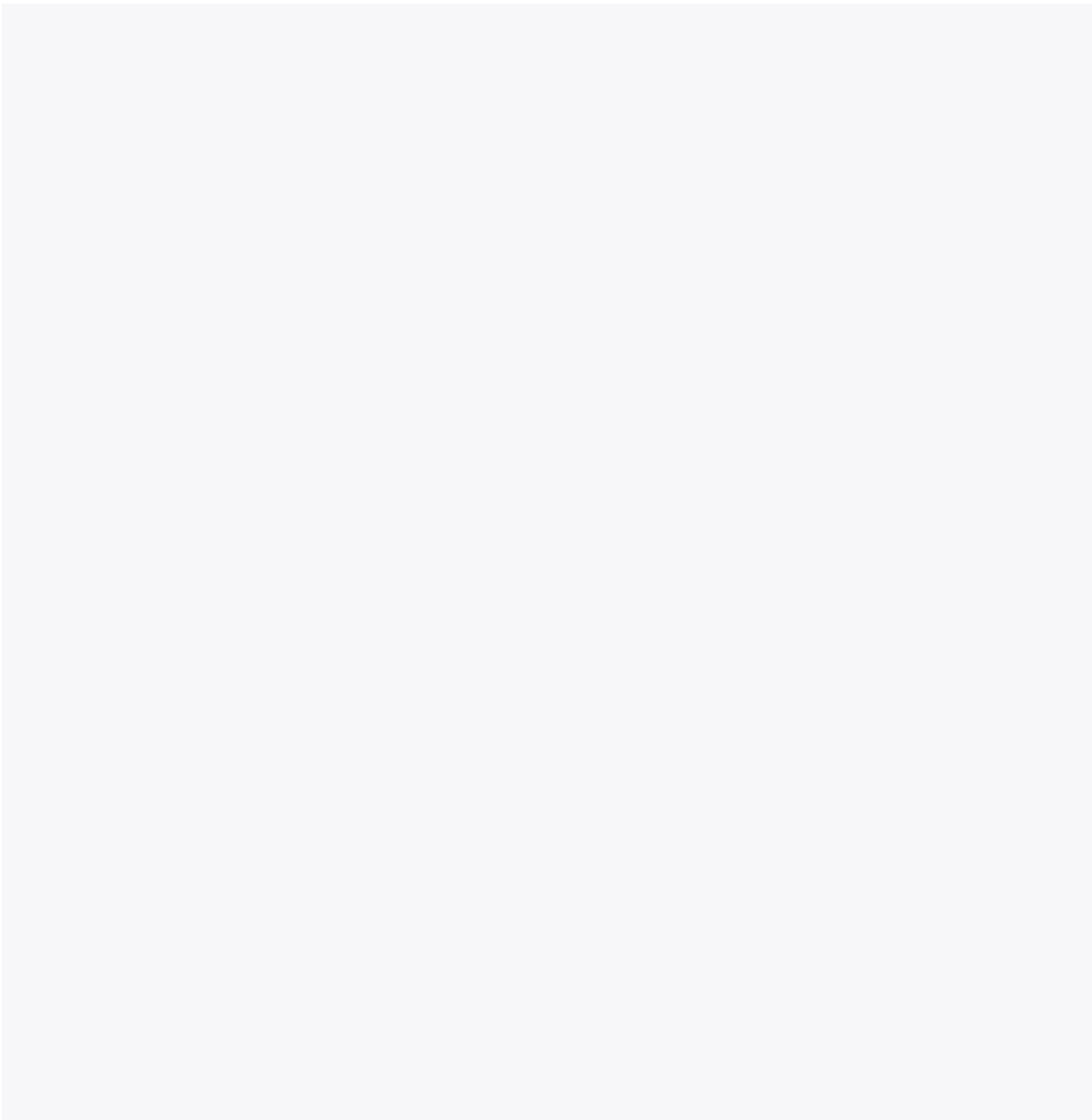
3.3 Organisation Interne d'une Classe C#

Une classe lisible regroupe ses membres selon un ordre standardisé reconnu par la communauté.

Tableau : Ordre d'agencement recommandé des membres d'une classe

Ordre	Catégorie de Membre	Description
1	Constantes	et <code>.</code>
2	Champs privés	Champs d'instance d'encapsulation <code>()</code> .
3	Constructeurs	Initialisateurs d'instance.
4	Propriétés publiques	Accesseurs de données <code>(, ,)</code> .
5	Méthodes publiques	Points d'entrée de la logique métier.
6	Méthodes privées	Méthodes utilitaires internes d'assistance.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39



Listing 12 – Exemple de structure canonique d'un service C#

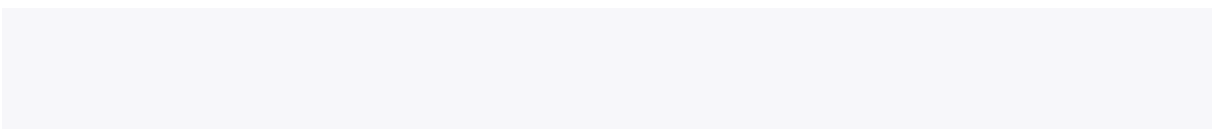
3.4 Inférence de Type () vs Type Explicite

Le mot-clé `infer` (introduit en C# 3) permet l'inférence de type à la compilation. Son utilisation est encadrée par des critères d'évidence visuelle.

Règle d'or de l'inférence

Utiliser `infer` uniquement lorsque le type retourné est **immédiatement explicite** à la lecture du côté droit de l'affectation (opérateur ou littéral). Dans le cas contraire, conserver le type explicite pour maintenir la clarté.

1
2
3
4



5
6
7
8
9
10

Listing 13 – Usage correct et incorrect de var

3.5 Contrôle de Flux et Retours Anticipés (*Early Return*)

Pour éviter l’empilement excessif de conditions imbriquées (syndrome de la pyramide d’indentation), C# préconise la technique des ****retours anticipés****.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33

Listing 14 – Réfactorisation d’une méthode complexe par Early Return

3.6 Gestion Rigoureuse des Exceptions

La gestion des erreurs doit respecter la chaîne de rétropropagation des exceptions sans altérer la pile d’appels (*stack trace*).

Règle d'or : `vs`

Lors du re-lancement d'une exception dans un bloc , utiliser strictement `vs` sans argument. L'instruction `vs` réinitialise la pile d'appels au niveau du `catch`, masquant l'origine exacte du plantage.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

Listing 15 – Bonnes pratiques de re-lancement d'exceptions

3.7 Outillage Automatisé (& Roslyn)

Afin de garantir le respect des conventions sans effort manuel, l'écosystème .NET utilise des fichiers de configuration standardisés à la racine du dépôt.

3.7.1 Fichier de Configuration

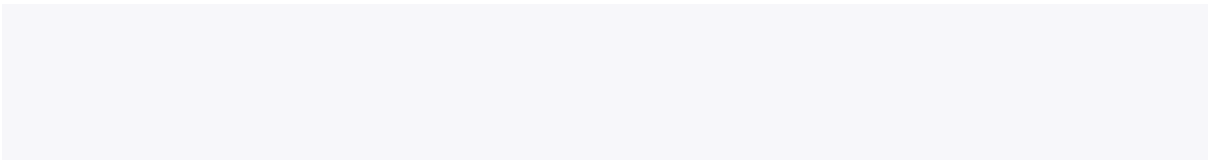
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20

Listing 16 – Exemple de fichier `.editorconfig` standard pour projet C#

3.7.2 Commande de Formatage CLI

La CLI .NET intègre un outil natif pour corriger le formatage du projet entier :

1
2
3
4
5



Listing 17 – Commandes dotnet format

3.8 Synthèse des Conventions

Tableau : Tableau récapitulatif des règles d'ingénierie C#

Élément	Convention Officielle	Exemple Validé
Classe / Structure	PascalCase	
Interface	I + PascalCase	
Méthode / Propriété	PascalCase	
Variable locale	camelCase	
Champ privé	_camelCase	
Accolades	Style Allman	Première ligne dédiée
Re-throw exception		Préserve la pile d'appels

4 Architecture des Types de Variables et Mémoire

En programmation C#, chaque variable possède un **type** statique fondamental qui définit non seulement l'espace mémoire alloué, mais également la nature des opérations permises par le processeur. Comprendre la dichotomie entre les types valeur et les types référence est essentiel pour maîtriser la gestion mémoire du Common Language Runtime (CLR).

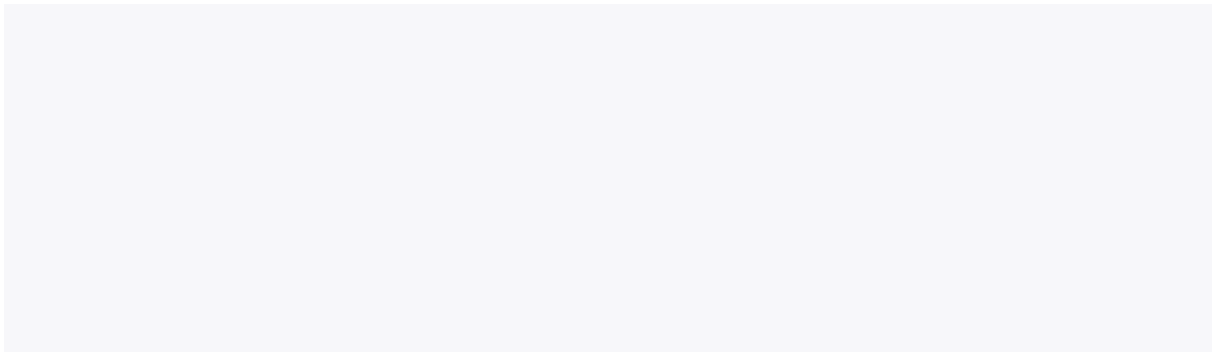
4.1 Types Valeur et Types Référence

Le CLR divise la mémoire de l'application en deux zones distinctes : la **Pile d'exécution** (*Stack*) et le **Tas managé** (*Managed Heap*). La classification d'un type détermine la zone d'allocation de la donnée.

4.1.1 Les Types Valeur (*Value Types*)

Les types valeur stockent directement leur contenu (la donnée brute) dans la zone mémoire qui leur est allouée sur la **pile** (*Stack*). Lorsqu'un type valeur est assigné à une nouvelle variable, le système effectue une **copie complète** de la valeur. Les deux variables opèrent alors sur des emplacements mémoire strictement indépendants.

1
2
3
4
5
6
7
8
9
10
11

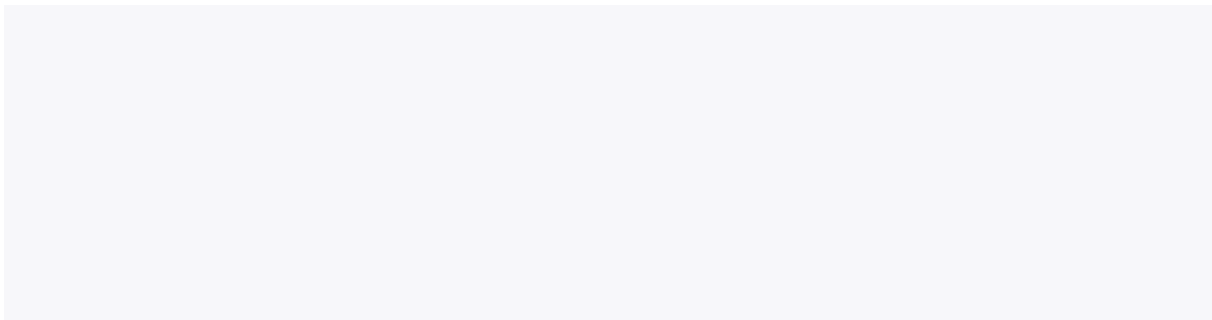


Listing 18 – Comportement en mémoire des types valeur (pile)

4.1.2 Les Types Référence (*Reference Types*)

Les types référence ne contiennent pas directement la donnée. Ils stockent une **adresse mémoire** (un pointeur managé) pointant vers l'objet réel alloué dans le **Tas** (*Heap*). Lors de l'affectation d'un type référence à une autre variable, seule l'adresse mémoire est copiée. Les deux variables pointent alors vers la **même instance** sous-jacente.

1
2
3
4
5
6
7
8
9
10



Listing 19 – Comportement en mémoire des types référence (tas)

Piège de l'ingénierie : L'Exception de Référence Nulle

Tenter d'accéder aux membres d'un type référence qui ne pointe vers aucune instance (état `nullptr`) provoquera l'arrêt brutal du programme via une `std::runtime_error`. Il est impératif d'initialiser les instances ou de vérifier la nullité avant accès.

4.2 Familles de Types Primitifs

4.2.1 Les Types Entiers (`int`, `short`, `long`)

Les entiers stockent des nombres sans composante fractionnaire. Le choix du type dépend de l'espace mémoire souhaité et des limites mathématiques attendues.

- `uint8_t` : Entier non signé de 8 bits (0 à 255). Hautement optimisé pour la manipulation de flux binaires ou le traitement de trames réseau.
- `int32_t` : Entier signé de 32 bits (-2×10^9 à 2×10^9). Le type standard par défaut pour les boucles et les compteurs.
- `int64_t` : Entier signé de 64 bits. Indispensable pour l'architecture Big Data, les identifiants de bases de données massives ou le calcul astronomique (suffixe `ll` recommandé).

L'Overflow (Dépassement de Capacité)

Assigner une valeur dépassant la limite physique du type (ex : assigner 256 à un `uint8_t`) provoque un *Overflow*. Selon la configuration du compilateur, cela entraînera une `std::overflow_error` ou un comportement cyclique silencieux entraînant des bugs critiques dans la logique d'état.

4.2.2 Les Types Flottants (`float`, `double`)

Les types à virgule flottante implémentent la norme IEEE 754 pour stocker des valeurs décimales. La précision de l'ingénierie détermine le choix du type.

- `float` (32-bit, suffixe `f`) : Optimisé pour les calculs vectoriels, la physique 3D et le rendu GPU où la vitesse de calcul prime sur l'exactitude extrême au micron près.
- `double` (64-bit) : Le standard C# par défaut pour les calculs scientifiques et géométriques.
- `long double` (128-bit, suffixe `l`) : Implémentation en base 10. Ce type requiert plus de cycles CPU mais évite toute aberration d'arrondi binaire. Son usage est **strictement obligatoire** pour les transactions financières et monétaires.

4.2.3 Les Types Textuels (`std::string` et `std::wstring`)

- `std::string` : Type valeur représentant un unique caractère Unicode UTF-16 (encadré par des guillemets simples `' '`).
- `std::wstring` : Type référence immuable représentant une séquence de caractères (encadrée par des guillemets doubles `" "`). Toute opération de modification (concaténation) alloue secrètement un nouvel objet en mémoire géré par le Garbage Collector.

L'ingénierie logicielle moderne favorise l'utilisation de l'**Interpolation de chaînes** (`std::string_view`) pour des raisons de lisibilité et de maintenance, évitant ainsi les concaténations verbeuses.

1
2
3
4
5
6

Listing 20 – Interpolation de chaînes de caractères

4.3 L'État Indéfini : Les Types Nullable

Les types valeur (`Value`) ne peuvent, par définition structurelle, pas contenir `Null`. Or, dans l'interaction avec des bases de données relationnelles ou des API JSON, une donnée peut légitimement être absente. C# permet d'envelopper un type valeur dans une structure autorisant le vide via l'opérateur suffixe `?`.

1
2
3
4
5
6
7
8

Listing 21 – Types nullable et opérateur de coalescence

4.4 Conversions Stratégiques : Le Parsing

Il est très fréquent d'ingérer des flux textuels (`Text`) nécessitant une transformation en types mathématiques exploitables. L'utilisation de `TryParse` est la norme de l'industrie pour prévenir les crashes liés aux entrées utilisateur malformées.

1
2
3
4
5
6
7
8
9
10
11
12

Listing 22 – Conversion sécurisée via TryParse

4.5 Synthèse du Chapitre

Tableau : Rétrospective de l'architecture des types

Concept	Règle de l'Ingénierie
Stack (Pile)	Zone mémoire ultra-rapide hébergeant les <i>Types Valeur</i> .
Heap (Tas)	Zone mémoire gérée par le GC hébergeant les instances des <i>Types Référence</i> .
	Le seul type flottant autorisé pour les opérations financières.
	Approche architecturale requise pour toute conversion d'entrée externe incertaine.

5 Les Opérateurs et Expressions

En C#, les opérateurs sont des symboles syntaxiques spécifiques qui indiquent au compilateur de réaliser des opérations mathématiques, logiques ou d’affectation sur des opérandes (variables ou valeurs). La maîtrise des opérateurs est indispensable pour manipuler les données et contrôler le flux d’exécution.

5.1 Opérateurs Arithmétiques

Les opérateurs arithmétiques effectuent des calculs mathématiques standard. Le comportement de la division dépend du type de ses opérandes : si les deux sont des entiers, le résultat est une division entière (tronquée).

Tableau : Opérateurs Arithmétiques

Opérateur	Opération	Exemple	Résultat
	Addition		
	Soustraction		
	Multiplication		
	Division entière		
	Modulo (reste)		

1
2
3
4

Listing 23 – Division entière vs Division flottante

5.2 Opérateurs d’Affectation

L’opérateur d’affectation de base est le signe `.` C# propose également des opérateurs d’affectation composés, qui combinent une opération arithmétique ou binaire avec l’affectation, permettant d’alléger la syntaxe.

Tableau : Opérateurs d’Affectation Composés

Opérateur	Syntaxe équivalente	Exemple
-----------	---------------------	---------

5.3 Opérateurs d’Incrémentement et Décrémentement

Ces opérateurs unaires augmentent ou diminuent la valeur d’une variable numérique de `.` L’emplacement de l’opérateur (préfixe ou suffixe) détermine le moment de l’évaluation par le CLR.

- **Préfixé** (`++`) : La variable est incrémentée **avant** l’évaluation de l’expression.
- **Suffixé** (`++`) : La variable est évaluée dans l’expression courante, puis incrémentée **après**.

1
2
3

Listing 24 – Incrémentation préfixée et suffixée

5.4 Opérateurs de Comparaison (Relationnels)

Ils évaluent la relation entre deux opérandes et retournent systématiquement une valeur de type `bool` (ou `false`). Ils constituent la base des structures conditionnelles.

Tableau : Opérateurs de Comparaison

Opérateur	Signification	Exemple
<code>==</code>	Égal à	
<code>!=</code>	Différent de	
<code>></code>	Strictement supérieur	
<code><</code>	Strictement inférieur	
<code>>=</code>	Supérieur ou égal	
<code><=</code>	Inférieur ou égal	

Erreur Courante : Affectation vs Égalité

Une erreur classique d'ingénierie consiste à utiliser l'opérateur d'affectation `=` (un seul signe égal) au lieu de l'opérateur de comparaison `==` dans une condition. Le compilateur C# bloquera cette erreur () si le résultat n'est pas un booléen.

5.5 Opérateurs Logiques

Les opérateurs logiques permettent de combiner plusieurs expressions booléennes. C# implémente l'évaluation "court-circuit" : si le premier opérande suffit à déterminer le résultat final, le second opérande n'est pas évalué.

Tableau : Opérateurs Logiques

Opérateur	Nom	Comportement Court-Circuit
<code>&&</code>	ET Logique (<i>AND</i>)	Si l'opérande gauche est <code>false</code> , l'opérande droit est ignoré (le résultat sera <code>false</code>).
<code> </code>	OU Logique (<i>OR</i>)	Si l'opérande gauche est <code>true</code> , l'opérande droit est ignoré (le résultat sera <code>true</code>).
<code>!</code>	NON Logique (<i>NOT</i>)	Inverse la valeur booléenne (opérateur unaire).

1
2
3
4
5
6
7
8

Listing 25 – Évaluation logique conditionnelle

5.6 Opérateurs Avancés et Bit à Bit (*Bitwise*)

Les opérateurs de bits manipulent directement la représentation binaire (les bits) d'une donnée en mémoire. Bien qu'essentiels en cryptographie, programmation réseau ou systèmes embarqués, ils sont rarement utilisés dans des applications de gestion standard.

- **ET bit à bit** (`&`) : si les deux bits sont à 1.
- **OU bit à bit** (`|`) : si au moins un des bits est à 1.
- **OU exclusif** (`^`) : si les bits sont différents.
- **Décalage** (`>>` / `<<`) : Décale les bits à gauche (multiplie par 2) ou à droite (divise par 2).

5.7 Opérateurs Spécifiques au Typage et Contrôle

C# intègre des opérateurs haut niveau pour gérer les types et l'état.

- **Opérateur ternaire** (`?:`) : Raccourci syntaxique pour une condition sur une seule ligne. Syntaxe : `condition ? valeur_si_vrai : valeur_si_faux`.
- **Opérateur de coalescence** (`??`) : Retourne l'opérande de gauche s'il n'est pas `null`, sinon retourne l'opérande de droite.
- **Opérateur `is`** : Évalue si un objet est compatible avec un type donné, au moment de l'exécution (Runtime).
- **Opérateur `as`** : Tente un transtypage (cast) en toute sécurité. Si l'opération échoue, retourne `null` au lieu de lever une exception.

1
2
3
4
5
6
7
8
9
10
11

Listing 26 – Utilisation de l'opérateur `is` et `as`

5.8 Synthèse du Chapitre

Tableau : Récapitulatif des opérateurs critiques

Concept	Règle de l'Ingénierie
Division entière	donne toujours <code>int</code> . Transtyper en <code>double</code> pour une précision flottante.
Court-circuit (<code>&&</code> , <code>&&&</code>)	Optimise les performances CPU en annulant l'évaluation de la partie droite si le résultat final est déjà connu.
et	Outils indispensables pour la vérification dynamique et le transtypage sécurisé sans levée d'exceptions bloquantes.

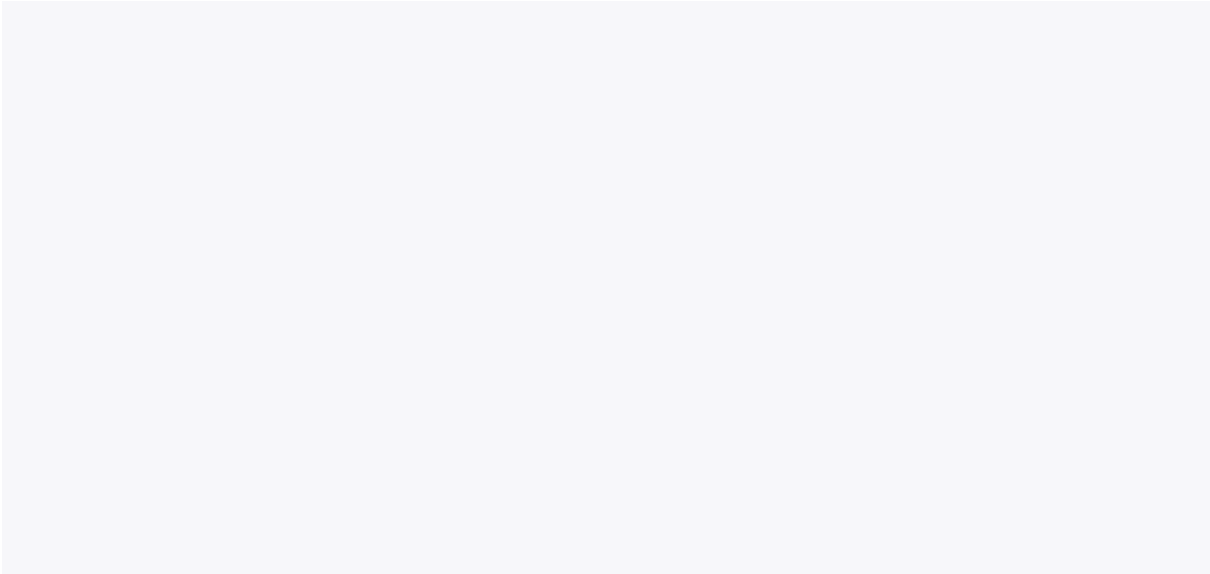
6 Structures Conditionnelles

Les structures conditionnelles, ou structures de contrôle de flux, permettent de dévier l'exécution linéaire d'un programme en fonction de l'évaluation d'expressions booléennes (vrai ou faux). Ces structures constituent le cœur de la logique décisionnelle algorithmique.

6.1 L'instruction `/`

L'instruction `/` évalue une condition. Si celle-ci retourne `true`, le bloc de code associé est exécuté. Les clauses `else` et `else if` permettent de traiter des alternatives exclusives, optimisant le processus car l'évaluation s'arrête dès qu'une condition est remplie.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18



Listing 27 – Logique décisionnelle en cascade

Règle d'ingénierie : Blocs d'instructions (Accolades)

Bien que le langage C# autorise l'omission des accolades si le bloc ne contient qu'une seule instruction, les conventions strictes (*Clean Code*) exigent de toujours les inclure. Cela prévient les bugs critiques de portée (*scope*) lors d'ajouts ultérieurs de code.

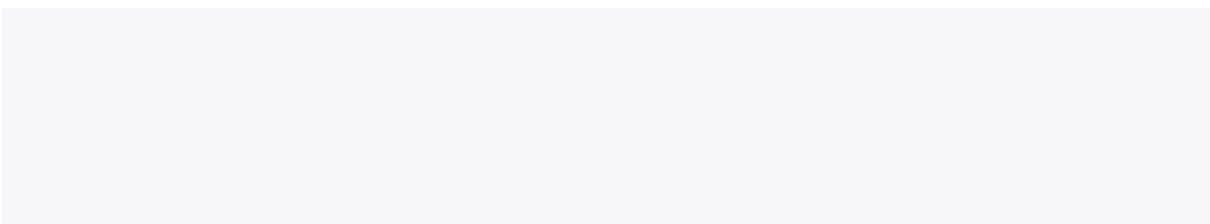
6.2 L'instruction `switch`

Le `switch` est utilisé pour évaluer une unique expression par rapport à une liste de modèles (*patterns*) ou de constantes. Il est structurellement plus lisible qu'une longue chaîne de `if` lors de la comparaison d'une même variable contre de multiples valeurs.

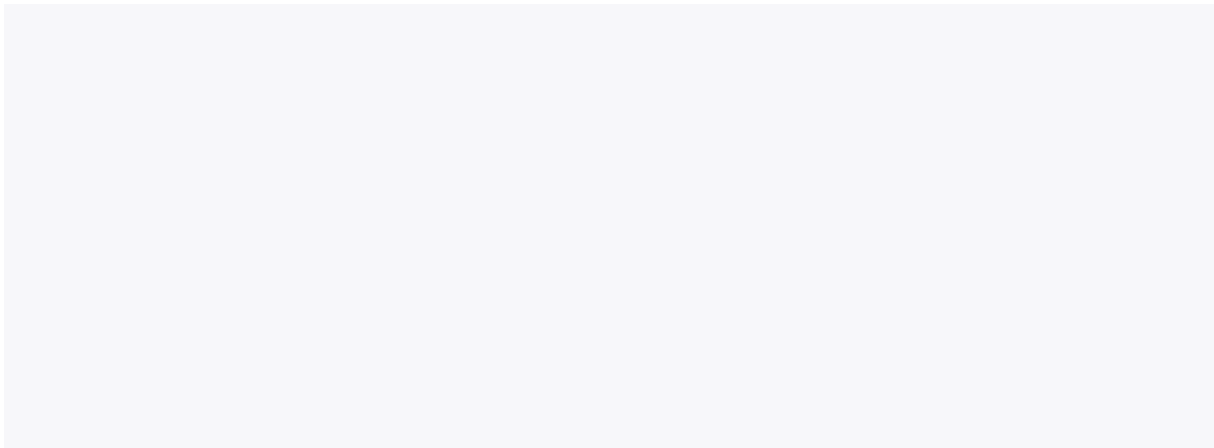
6.2.1 Syntaxe classique

Dans sa forme traditionnelle, chaque cas `case` doit se terminer obligatoirement par l'instruction `break` pour empêcher le compilateur de continuer l'exécution sur le cas suivant (erreur `fall-through`).

1
2
3
4
5
6
7



8
9
10
11
12
13
14
15
16
17
18
19
20
21

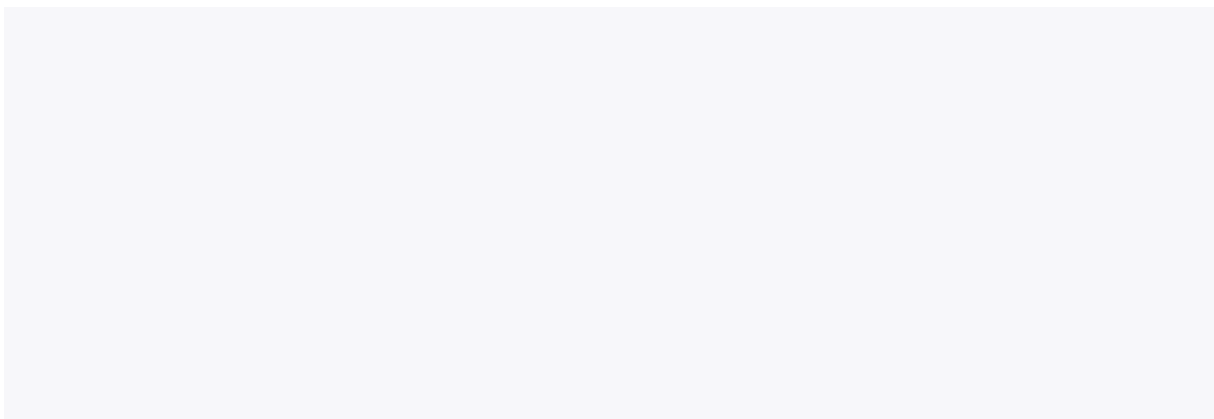


Listing 28 – Routage par instruction switch

6.2.2 Les Expressions (C# 8.0+)

Le C# moderne a introduit les **expressions** , qui transforment la structure de contrôle classique en une instruction retournant directement une valeur. La syntaxe remplace par l’opérateur ”flèche” et par le rejet (*discard*) .

1
2
3
4
5
6
7
8
9
10
11
12
13



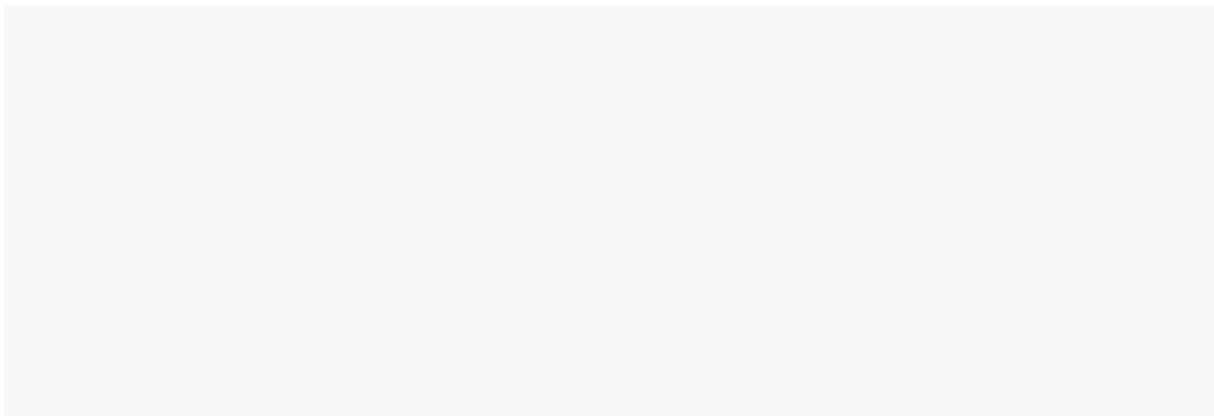
Listing 29 – Affectation via expression switch moderne

6.3 L’opérateur Ternaire Conditionnel ()

L’opérateur ternaire condense une affectation conditionnelle simple sur une seule ligne de code. Il évalue une expression booléenne, puis retourne l’une des deux valeurs proposées.

Syntaxe :

1
2
3
4
5
6
7
8
9
10
11
12
13



14
15

Listing 30 – Optimisation par opérateur ternaire

Anti-pattern : Les ternaires imbriqués

L'imbrication d'opérateurs ternaires (ex :) détruit la lisibilité du code. L'ingénierie logicielle proscriit formellement cette pratique au profit d'un standard ou d'une expression .

6.4 Synthèse du Chapitre

Tableau : Rétrospective des structures de contrôle

Structure	Cas d'usage architectural
	Évaluation de conditions hétérogènes complexes (ex : plages de valeurs, combinaison de variables).
classique	Exécution de blocs d'instructions distincts basés sur l'état d'une variable unique.
expression	Affectation directe et fonctionnelle d'une variable basée sur de multiples valeurs statiques.
Ternaire ()	Affectation binaire rapide sur une seule ligne (à condition stricte de non-imbrication).

7 Les Boucles (Structures Itératives)

Les boucles, ou structures itératives, sont des mécanismes de contrôle de flux permettant d'exécuter un bloc d'instructions de manière répétée tant qu'une condition booléenne demeure évaluée à `true`. Elles sont indispensables en ingénierie logicielle pour l'automatisation de tâches répétitives, le parcours de structures de données ou l'attente d'événements asynchrones.

7.1 La boucle `do while`

La boucle `do while` est privilégiée lorsque le nombre d'itérations est connu à l'avance ou déterminé par une borne fixe. Elle encapsule l'initialisation de l'index, la condition de maintien et l'incrément (ou décrémentation) dans une seule ligne de déclaration.

1
2
3
4

Analyse de l'exécution :

- **Initialisation** (`init`) : Exécutée une seule fois au démarrage de la boucle.
- **Condition** (`cond`) : Évaluée avant chaque itération. Si elle est `true`, la boucle s'arrête et le programme continue à l'instruction suivante.
- **Incrément** (`incr`) : Exécutée à la fin de chaque itération, juste avant de réévaluer la condition.

7.2 La boucle `while`

La boucle `while` évalue sa condition de maintien *avant* d'exécuter son bloc d'instructions. Elle est typiquement employée lorsque le nombre d'itérations est indéterminé, dépendant d'un état dynamique du système ou de la lecture d'un flux de données (ex : lecture de fichiers, écoute réseau).

1
2
3
4
5
6

Contrairement au `do while`, l'initialisation de la variable de contrôle est externe à la boucle, et son altération doit obligatoirement figurer à l'intérieur du bloc pour éviter une boucle infinie.

7.3 La boucle `do`

Contrairement à la boucle `while`, la variante `do` évalue la condition *après* l'exécution du bloc. Cela garantit que les instructions internes seront exécutées au strict minimum une fois, indépendamment de l'état initial.

1
2
3
4
5
6

Cette boucle est très utile pour les scénarios nécessitant au moins une action initiale, tels que les tentatives de connexion (*retry policy*) ou l'exécution d'un menu interactif.

7.4 La boucle

La structure `for` offre une syntaxe abstraite et sécurisée pour itérer sur l'ensemble des éléments d'une collection (tableaux, listes) implémentant l'interface `Iterable`. Elle élimine la gestion manuelle de l'index et prévient les erreurs de dépassement.

1
2
3
4
5

Bien que cette boucle produise un code plus concis et robuste, la variable d'itération (dans l'exemple) est en lecture seule. Vous ne pouvez pas modifier directement l'élément du tableau via cette variable.

7.5 Contrôle de Flux : `break` et `continue`

Au sein de toute structure itérative, il est possible d'altérer le flux standard à l'aide de deux mots-clés :

- `break` : Interrompt immédiatement et définitivement la boucle englobante, forçant le programme à passer à l'instruction qui suit la boucle.
- `continue` : Interrompt l'itération courante et passe directement à l'évaluation de la condition pour démarrer l'itération suivante.

Pièges Courants & Erreurs

- **La boucle infinie** : Oublier d'incrémenter la variable d'arrêt ou de modifier la condition dans une boucle `for`, entraînant le blocage complet du thread (*Thread Starvation*).
- **Modification de la collection** : Tenter d'ajouter ou de supprimer un élément d'une liste qui est actuellement en train d'être parcourue par un `for` déclenche inmanquablement une `ConcurrentModificationException` à l'exécution.
- **Erreur de borne (*Off-by-one error*)** : Utiliser `array.length` au lieu de `array.length - 1` lors du parcours d'un tableau classique avec une boucle `for`. Puisque les tableaux sont indexés à partir de 0, cela mène à une `ArrayIndexOutOfBoundsException`.

7.6 Synthèse

- `for` : À privilégier pour les itérations indexées dont le nombre est prévisible et fini.
- `while` : Idéal pour les processus en attente d'un état ou lisant un flux de données continu.
- `do-while` : Utile lorsqu'un premier traitement est impératif avant toute vérification de condition.
- `forEach` : La méthode la plus élégante et robuste pour parcourir des structures de données en toute sécurité.

8 Tableaux et Collections de base

En ingénierie logicielle, la capacité à regrouper et manipuler efficacement de multiples données connexes est primordiale. C# propose plusieurs structures fondamentales : depuis les tableaux primitifs à taille fixe jusqu'aux collections génériques dynamiques fournies par le framework .NET. Le choix de la bonne structure influence directement les performances en termes d'allocation mémoire et de rapidité d'exécution.

8.1 Les Tableaux ()

Les tableaux représentent la structure la plus bas niveau du langage. Ils stockent des éléments de même type de manière contiguë en mémoire. Leur caractéristique principale est leur **taille fixe**, définie obligatoirement au moment de l'instanciation.

1
2
3
4
5
6
7
8
9
10
11
12

L'accès à un élément par son index s'effectue en temps constant $O(1)$. Cependant, la méthode crée en réalité un *nouveau* tableau en mémoire et y copie tous les éléments de l'ancien. Cette opération est très coûteuse en performance, c'est pourquoi les tableaux ne doivent être utilisés que lorsque la quantité d'éléments est connue et définitive.

8.2 Les Listes ()

Pour pallier la rigidité des tableaux, le framework .NET fournit la collection générique `List`. Elle encapsule un tableau sous-jacent et gère automatiquement et silencieusement son redimensionnement lors de l'ajout ou de la suppression d'éléments.

1
2
3
4
5
6
7
8
9
10
11
12
13
14

La `List` est la structure par défaut recommandée en C# pour stocker des données séquentielles nécessitant de la flexibilité.

8.3 Les Dictionnaires ()

Le dictionnaire implémente une table de hachage (*Hash Table*). Il stocke des données sous forme de paires "Clé/Valeur". La recherche d'une valeur à partir de sa clé s'effectue via un calcul de hachage, garantissant un temps de recherche proche de $O(1)$.

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Le dictionnaire est incontournable lorsque les données doivent être indexées et retrouvées via un identifiant métier (ex : un matricule employé, un code produit) plutôt que par une simple position ordinale.

8.4 Les Énumérations ()

Bien qu'il ne s'agisse pas d'une collection à proprement parler, l' est un type fondamental permettant de définir un ensemble de constantes liées. Il améliore drastiquement la lisibilité du code en remplaçant les entiers bruts (souvent qualifiés de "nombres magiques") par des identifiants sémantiques clairs.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

Pièges Courants & Erreurs

- **IndexOutOfRangeException** : Essayer d'accéder à sur un tableau de taille 5 (les index valides vont de 0 à 4 puisque indexés à partir de 0).
- **KeyNotFoundException** : Tenter de lire une valeur dans un dictionnaire avec une clé inexistante (ex :). Si l'existence de la clé est incertaine, utilisez plutôt .
- **ArgumentException (Clé doublée)** : Utiliser la méthode d'un dictionnaire avec une clé qui y est déjà présente. Un dictionnaire exige des clés strictement uniques.

8.5 Fonctions de Manipulation Avancées

Le framework .NET propose de nombreuses fonctions utilitaires pour interagir avec les collections sans réinventer la roue.

8.5.1 Utilitaires pour Tableaux et Listes

1
2
3
4
5
6
7
8
9
10
11
12
13

8.5.2 Utilitaires pour Dictionnaires

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19

8.6 Synthèse

- **Tableau ()** : Taille fixe, bas niveau, performant. Idéal pour les données immuables.
- **Liste ()** : Taille dynamique, encapsulation intelligente. Idéale pour les collections appelées à évoluer en nombre.
- **Dictionnaire ()** : Recherche instantanée par clé métier. L'arme absolue pour retrouver une donnée précise sans parcourir toute la collection.
- **Énumération ()** : Permet de définir des états restreints de manière propre, typée et compréhensible.

9 Fonctions et Méthodes

En ingénierie logicielle, la création de méthodes (le terme C# pour désigner les fonctions rattachées à une classe) est fondamentale pour garantir la modularité, la réutilisabilité et la lisibilité du code. Une méthode regroupe un ensemble d'instructions accomplissant une tâche algorithmique ou métier spécifique.

9.1 Déclaration, Paramètres et Valeur de Retour

La signature d'une méthode définit son comportement externe : son niveau d'accès, son type de retour, son identifiant et ses paramètres d'entrée. Si la méthode ne produit aucune donnée de sortie exploitable, son type de retour est déclaré comme `void`.

1
2
3
4
5
6
7
8
9
10
11
12

La clause `return` stoppe immédiatement l'exécution de la méthode et restitue la valeur à l'appelant. Le type de cette valeur doit correspondre strictement à la signature définie.

9.2 Paramètres Optionnels et Surcharge

C# offre deux mécanismes distincts pour assouplir l'appel des méthodes :

9.2.1 Valeurs par défaut (Paramètres Optionnels)

Un paramètre peut se voir attribuer une valeur par défaut dans la signature. Il devient alors optionnel lors de l'appel. Les paramètres optionnels doivent obligatoirement être déclarés *à la fin* de la liste des paramètres.

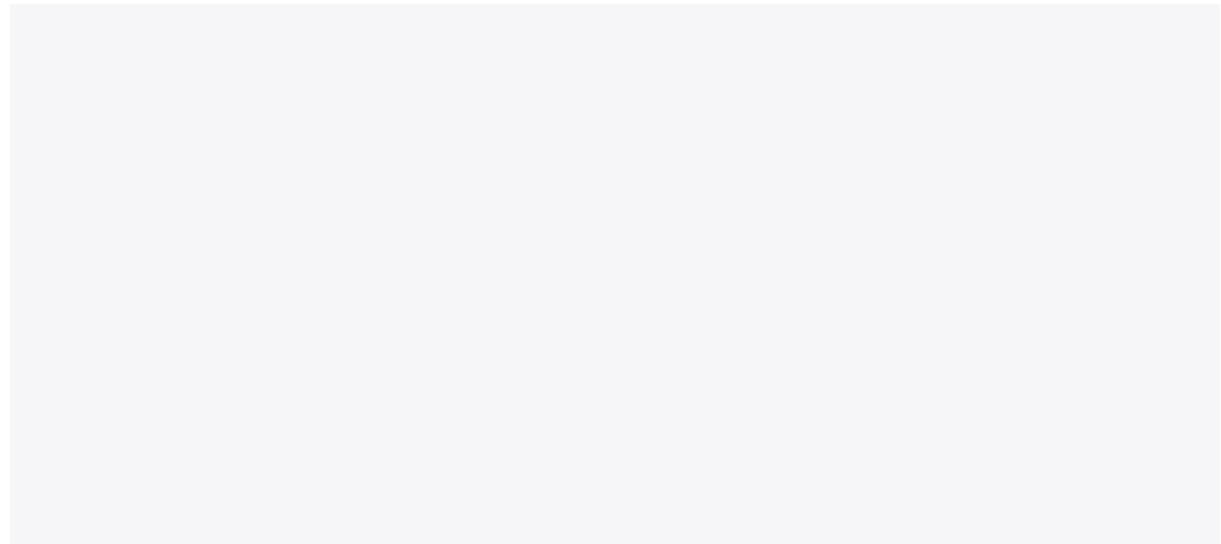
1
2
3
4
5
6
7
8

9.2.2 Surcharge de Méthodes (*Overloading*)

La surcharge consiste à définir plusieurs méthodes portant **le même nom**, mais ayant une **signature différente** (nombre, type ou ordre des paramètres). Le compilateur détermine la méthode à exécuter en analysant les arguments passés.

L'avantage principal de l'*overloading* est de proposer une API naturelle. Au lieu d'inventer des noms comme `Get` et `Set`, on centralise l'action sous un seul verbe.

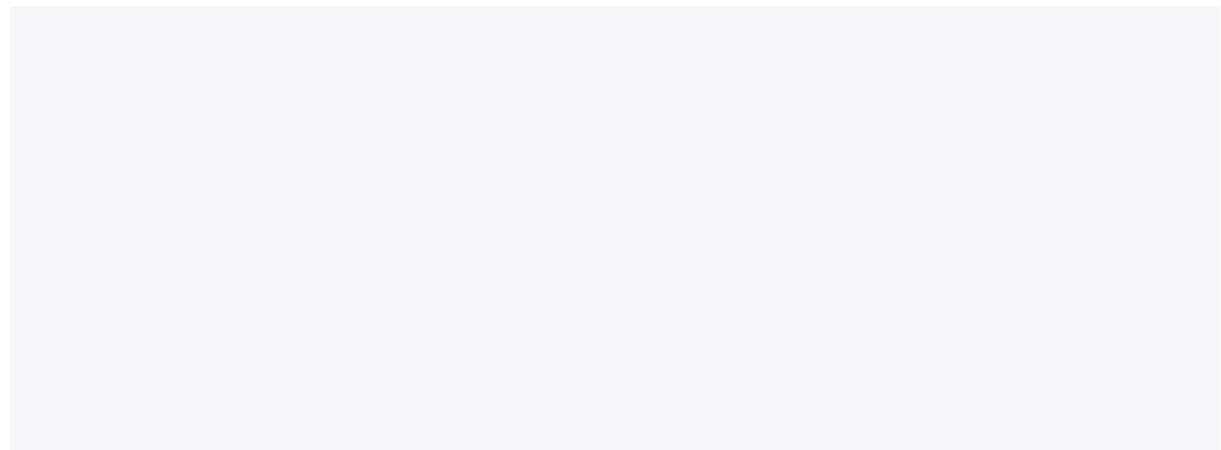
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17



9.3 Nombre Variable d'Arguments ()

Le mot-clé `...` permet à une méthode d'accepter un nombre indéfini d'arguments du même type, qui seront encapsulés automatiquement dans un tableau. Ce mot-clé ne peut être appliqué qu'au tout dernier paramètre de la signature.

1
2
3
4
5
6
7
8
9
10
11
12
13
14

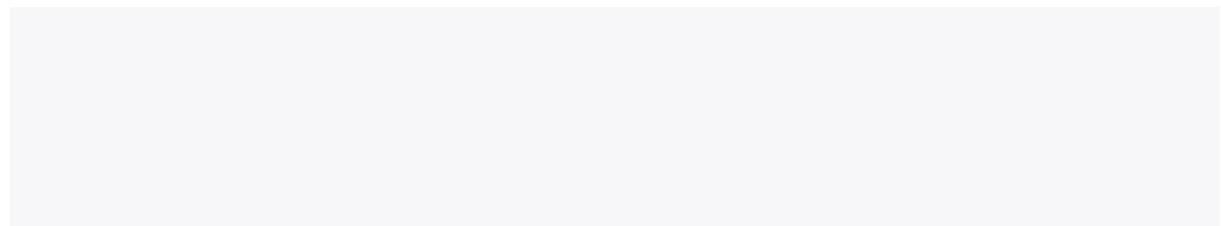


9.4 Passage de Paramètres : et

Par défaut, le passage d'arguments (pour les types valeur) s'effectue **par valeur** (copie isolée). Le modificateur `ref` exige un passage par référence (adresse mémoire), permettant à la méthode de muter directement la variable de l'appelant. La variable doit obligatoirement être initialisée avant l'appel.

Le modificateur `out` opère de la même manière, mais son rôle est d'exporter un résultat. Il exempte l'appelant d'initialiser la variable, mais force contractuellement la méthode à lui assigner une valeur avant l'instruction de retour.

1
2
3
4
5
6
7



*Note : Bien que fut longtemps utilisé pour retourner de multiples valeurs, l'architecture C# moderne préconise plutôt l'utilisation des **Tuples** pour cet usage.*

9.5 Méthodes Statiques vs d'Instance

- **Méthodes d'instance** : Nécessitent l'instanciation préalable de la classe (la création d'un objet en mémoire). Elles peuvent manipuler l'état interne (les champs) propre à cette instance spécifique.
- **Méthodes statiques ()** : Sont attachées au type lui-même (la classe) et non à une instance. Elles ne peuvent pas accéder aux données d'une instance. Elles sont conçues pour des opérations utilitaires pures (ex : ou).

Pièges Courants & Erreurs

- **Ambiguïté de surcharge** : Créer deux méthodes avec des signatures identiques mais des types de retour différents génère une erreur de compilation, car le type de retour ne fait pas partie de la signature permettant de résoudre la surcharge.
- **Rupture du contrat** : survient inévitablement si la méthode omet d'affecter une valeur au paramètre avant de se terminer.
- **Ordre des paramètres optionnels** : Tenter de définir un paramètre optionnel *avant* un paramètre requis provoquera une erreur stricte du compilateur.

9.6 Synthèse

- / **Type de retour** : Définit si la méthode opère comme une action silencieuse ou comme une fonction calculant un résultat.
- **Surcharge & Paramètres optionnels** : Flexibilisent l'API de votre classe et facilitent l'appel de vos méthodes.
- : Simplifie le passage d'une liste dynamique de variables du même type.
- / : Permettent le passage par référence, utiles pour muter la variable appelante ou implémenter le pattern .
- : Rend la méthode globale et indépendante de tout état d'instance.

10 Manipulation des Types de Base (Chaînes et Nombres)

En C#, le type `string` (alias de la classe `String`) représente une séquence de caractères Unicode. Une caractéristique fondamentale à assimiler en ingénierie logicielle est l'**immutabilité** des objets : une fois créée en mémoire, une chaîne de caractères ne peut plus être modifiée. Toute opération de transformation (mise en majuscules, remplacement, découpage) instancie et retourne une *nouvelle* chaîne en mémoire, laissant la chaîne d'origine intacte.

10.1 Propriétés et Méthodes de Base

1
2
3
4
5
6
7
8
9
10
11

10.2 Recherche et Vérification

Pour vérifier le contenu d'une chaîne sans nécessairement l'extraire, C# propose des méthodes d'évaluation retournant un booléen ou un index.

1
2
3
4
5
6
7
8
9

10.3 Découpage, Concaténation et Remplacement

La manipulation de données brutes (comme des fichiers CSV ou des logs) requiert souvent d'extraire, d'isoler ou de remplacer des segments de texte.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

10.4 Insertion et Suppression

1
2
3
4
5
6
7
8
9

10.5 Interpolation et Optimisation ()

10.5.1 Interpolation de chaînes ()

L'interpolation, introduite en C# 6, permet d'injecter directement des expressions ou variables dans une chaîne à l'aide d'accolades . C'est l'approche la plus lisible pour la construction de chaînes dynamiques, bien supérieure à la concaténation par l'opérateur .

1
2
3
4

10.5.2 Optimisation avec

À cause de l'immutabilité du type , une boucle qui concatène des milliers de chaînes via provoquera autant d'allocations mémoires superflues, saturant le *Garbage Collector*. Dans les contextes de forte performance ou de modifications intensives, il est impératif d'utiliser la classe (namespace).

1
2
3
4
5
6
7
8
9

Pièges Courants & Erreurs

- **Immutabilité ignorée** : Écrire `sb.Append(s)` seul sans récupérer le résultat `sb` n'aura aucun effet sur la variable `sb`.
- **NullReferenceException** : Appeler `sb.ToString()` ou `sb.ToStringBuilder()` sur une variable valant `null` provoquera un crash instantané. Il faut toujours vérifier la nullité avec `sb != null` avant manipulation.
- **Concaténation dans des boucles** : Utiliser l'opérateur `+` sur une chaîne dans une boucle de plus d'une dizaine d'itérations est considéré comme un défaut d'architecture (*Code Smell*) pour les performances mémoire.

10.6 Manipulation des Nombres et Mathématiques

La classe statique `Math` fournit l'ensemble des opérations mathématiques fondamentales nécessaires en ingénierie logicielle. Parallèlement, les types primitifs numériques disposent de méthodes de parsing sécurisées.

1
2
3
4
5
6
7
8

10.6.1 Conversion et Parsing

La conversion d'une chaîne de caractères vers un entier est une opération risquée qui doit toujours être sécurisée.

1
2
3
4
5
6
7
8
9
10
11
12
13
14

10.7 Synthèse

- **Propriétés fondamentales** : Les objets `String` sont immuables. Chaque modification génère une nouvelle allocation mémoire.
- **Évaluation** : Utiliser `Integer.parseInt()` et `Long.parseLong()` pour analyser le contenu sans créer de nouvelles chaînes.
- **Formatage dynamique** : Privilégier systématiquement l'interpolation `String.format()` pour la lisibilité du code.
- **Performance** : Utiliser exclusivement `StringBuilder` lors de la construction itérative de chaînes dans des boucles.

11 Gestion des Erreurs et Exceptions

En ingénierie logicielle, un programme informatique ne s'exécute jamais dans un environnement parfait. Des anomalies externes (perte de réseau, fichier inexistant, base de données injoignable) ou internes (division par zéro, référence nulle) peuvent survenir. Le framework .NET gère ces anomalies via le concept d'**Exception**.

Une exception est un objet (héritant de la classe de base `Exception`) qui est "levé" (*thrown*) par le runtime lorsqu'une erreur critique empêche la poursuite normale de l'exécution. Si cette exception n'est pas "capturée" (*caught*) par le développeur, le programme s'arrête brutalement (*Crash*).

11.1 Le Bloc

L'interception des erreurs s'effectue via une structure de contrôle dédiée.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24

Il est primordial en C# de toujours ordonner les blocs `try` du type d'exception le plus précis (`ArgumentException`) vers le type le plus générique (`Exception`).

11.1.1 Exceptions courantes du Framework

- `ArgumentException` : Tentative d'accès à un membre d'un objet valant `0`.
- `IndexOutOfRangeException` : Accès à un index de tableau inexistant.
- `ParseException` : Échec lors du *parsing* d'un type (`int`).

11.2 Le Mot-Clé `throw` et la Propagation

Il est souvent nécessaire de déclencher volontairement une erreur lorsqu'une validation métier échoue. Le mot-clé `throw` interrompt immédiatement la méthode courante et remonte l'exception à l'appelant.

1
2
3

4
5
6
7
8
9
10

11.3 Création d'Exceptions Personnalisées

Pour modéliser des erreurs métiers spécifiques (ex : un solde insuffisant), il convient de créer ses propres classes d'exception héritant de .

1
2
3
4
5
6
7
8
9
10
11

11.4 Gestion des Ressources : Le Bloc

La libération des ressources dites "non-managées" (fichiers système, connexions réseaux, flux de base de données) ne doit pas dépendre du Garbage Collector. Le bloc garantit que la méthode de la ressource sera appelée à la fin du bloc, **même si une exception est levée** à l'intérieur.

1
2
3
4
5
6

Pièges Courants & Anti-Patterns

- **Le Catch Vide (*Swallowing*)** : Écrire un `try-catch` sans logger ou traiter l'erreur masque silencieusement les bugs critiques du programme.
- **Mauvaise Propagation ()** : Écrire `try-catch` dans un bloc écrase la trace d'appel (*Stack Trace*) d'origine. Il faut utiliser le mot-clé `throw` seul pour propager l'erreur sans la masquer.
- **Utiliser les exceptions pour le contrôle de flux** : Les exceptions coûtent extrêmement cher en temps de calcul CPU. Ne les utilisez jamais pour vérifier des conditions prévisibles (préférez `if` plutôt qu'un `try-catch` sur).

11.5 Synthèse

- `try-with-resources` : Encapsule le code susceptible d'échouer.
- `try-catch` : Intercepte et traite un type spécifique d'erreur.
- `try-catch-finally` : Assure l'exécution du code de nettoyage critique.

- : Déclenche manuellement ou propage une exception.
- : Remplace avantageusement un bloc `try` pour garantir la destruction des ressources non-managées (`using`).